Architecture — Appli pour musiciens (Cube 3D)

Oct 23, 2026 · @ben

Vue d'ensemble

L'application est un cube 3D dont chaque face est un outil pour musicien : accordeur, tablatures, métronome, dictionnaire d'accords, carnet d'entraînement et réglages. On fait pivoter le cube pour passer d'un outil à l'autre.

Le code est découpé en quatre couches qui ne se mélangent pas : la scène 3D (le cube), les outils (le contenu des faces), l'état global (ce qui est partagé) et les services (ce qui parle au micro, aux haut-parleurs ou à Internet). Ce découpage permet à chaque membre du binôme de travailler dans son dossier sans marcher sur les fichiers de l'autre.

Technologie	Rôle	Pourquoi ce choix
Vite	Serveur de dev et build	Démarrage instantané, rechargement à chaud, preset reconnu par Vercel
React	Interface en composants	Une face = un composant isolé, facile à répartir entre deux personnes
TypeScript	Typage du code (mode strict)	Les erreurs de type sont détectées avant l'exécution ; les interfaces (store, services) servent de contrat entre les deux membres du binôme
React Three Fiber + drei	Scène 3D en JSX	Écrire Three.js comme des composants React ; drei fournit `<Html>` pour projeter l'UI sur les faces et `OrbitControls`
Zustand	État global	Plus léger que Redux, pas de Provider à câbler, lisible depuis la scène 3D comme depuis l'UI
Web Audio API	Micro et son	Intégrée au navigateur, aucune dépendance : analyse du micro (accordeur) et synthèse (métronome)
Vercel	Hébergement	Déploiement automatique à chaque push sur `main`, variables d'environnement gérées en ligne

Arborescence

Le repo tient en un dossier `src/` (le code) entouré de quelques fichiers de configuration à la racine.

```
Appli-pour-musiciens-ue5-react/
├── .github/
│   └── pull_request_template.md
├── docs/
│   └── architecture.pdf
├── src/
│   ├── main.tsx
│   ├── App.tsx
│   ├── index.css
│   ├── vite-env.d.ts
│   ├── components/
```

```
|   |   ├── 3d/
|   |   |   ├── Scene.tsx
|   |   |   ├── Cube.tsx
|   |   |   └── CubeControls.tsx
|   |   └── tools/
|   |       ├── FaceTuner.tsx
|   |       ├── FaceTabs.tsx
|   |       ├── FaceMetronome.tsx
|   |       ├── FaceChordDictionary.tsx
|   |       ├── FacePracticeNotebook.tsx
|   |       └── FaceSettings.tsx
|   ├── store/
|   |   └── useAppStore.ts
|   └── services/
|       ├── audioEngine.ts
|       └── tablatureApi.ts
├── index.html
├── vite.config.ts
├── tsconfig.json
├── package.json
├── package-lock.json
├── .env.example
├── .gitignore
└── README.md
```

Non versionnés (ignorés par git) : `node_modules/` (régénéré par `npm install`), `dist/` (régénéré par `npm run build`) et `.env` (secrets).

Fichiers racine

Ces fichiers ne contiennent pas de fonctionnalité : ils disent comment le projet s'installe, se lance, se construit et se partage.

Fichier	Ce qu'il apporte	Pourquoi il est là
`package.json`	Liste des dépendances et des commandes `dev`, `build`, `typecheck`, `preview`	Point d'entrée de npm : sans lui, rien ne s'installe ni ne se lance
`package-lock.json`	Versions exactes installées	Garantit que les deux membres du binôme et Vercel ont exactement les mêmes versions

Fichier	Ce qu'il apporte	Pourquoi il est là
`tsconfig.json`	Réglages de TypeScript (mode strict, JSX React)	`npm run build` lance d'abord `tsc` : une erreur de type bloque le build, donc le déploiement
`vite.config.ts`	Configuration de Vite (plugin React)	Permet à Vite de compiler le TSX ; c'est ici qu'on ajoutera alias ou options de build
`index.html`	Page unique, contient `<div id="root">`	Vite part de ce fichier ; React s'y greffe via `src/main.tsx`
`.env.example`	Modèle des variables (URL et clé de l'API de tablatures)	Montre quelles variables remplir sans jamais publier la vraie clé
`.gitignore`	Fichiers exclus de git	Empêche de pousser `node_modules`, le build et le `.env` secret
`README.md`	Installation, workflow Git, tableau des rôles	Première page affichée sur GitHub, référence commune du binôme
`.github/pull_request_template.md`	Modèle pré-rempli de Pull Request	Chaque PR dit ce qu'elle change et comment la tester, ce qui facilite la relecture croisée

Le dossier src/

Chaque sous-dossier de `src/` répond à une seule question : à quoi ça ressemble en 3D, que fait chaque outil, qu'est-ce qui est partagé, qu'est-ce qui parle à l'extérieur.

Point d'entrée : main.tsx, App.tsx, index.css

- `main.tsx` monte React dans la page. On n'y touche quasiment jamais.
- `App.tsx` est le layout principal : il pose le `<Canvas>` 3D en plein écran et accueillera l'UI globale superposée (navigation, titre).
- `index.css` porte les styles globaux. Il est séparé pour ne pas mélanger style et logique.

- `vite-env.d.ts` déclare les types des variables d'environnement (`VITE_TABS_API_URL`, `VITE_TABS_API_KEY`). TypeScript signale ainsi une variable mal orthographiée.

components/3d/ : la scène et le cube

Tout ce qui vit dans l'espace 3D. Le dossier est isolé parce que ce code obéit aux règles de Three.js (positions, rotations, boucle d'animation), pas à celles d'une page web classique.

Fichier	Responsabilité
`Scene.tsx`	Racine de la scène : lumières, caméra, contrôles d'orbite, transitions entre les faces
`Cube.tsx`	Le modèle du cube et la projection de l'UI de chaque outil sur ses faces (drei `<Html>` ou textures)
`CubeControls.tsx`	L'interaction : rotation à la souris, pivot vers une face au clic

components/tools/ : une face = un outil

Six composants indépendants, un par face. Le cube les affiche sans savoir ce qu'ils font, et chacun ignore qu'il est sur un cube. On peut donc développer et tester un outil seul, puis le brancher sur sa face. C'est aussi l'unité naturelle pour se répartir le travail.

Fichier	Outil
`FaceTuner.tsx`	Accordeur guitare/ukulélé (détection de hauteur)
`FaceTabs.tsx`	Recherche et affichage de tablatures
`FaceMetronome.tsx`	Métronome : BPM et signature rythmique
`FaceChordDictionary.tsx`	Dictionnaire d'accords et positions sur le manche
`FacePracticeNotebook.tsx`	Carnet de notes et suivi des séances, sauvegardés dans le navigateur (localStorage, pas de back-end)
`FaceSettings.tsx`	Réglages : thème, sensibilité, instrument, audio

store/ : l'état partagé

`useAppStore.ts` (Zustand) garde ce que plusieurs parties doivent connaître en même temps : la face active (le cube et la navigation la lisent), l'état audio (métronome en marche, fréquence de l'accordeur), les préférences utilisateur et le cache des tablatures. Sa forme est décrite dans l'interface `AppState` : c'est le contrat que les deux membres du binôme partagent. Une donnée qui ne sert qu'à un seul outil reste dans le composant de cet outil, pas dans le store.

services/ : ce qui parle à l'extérieur

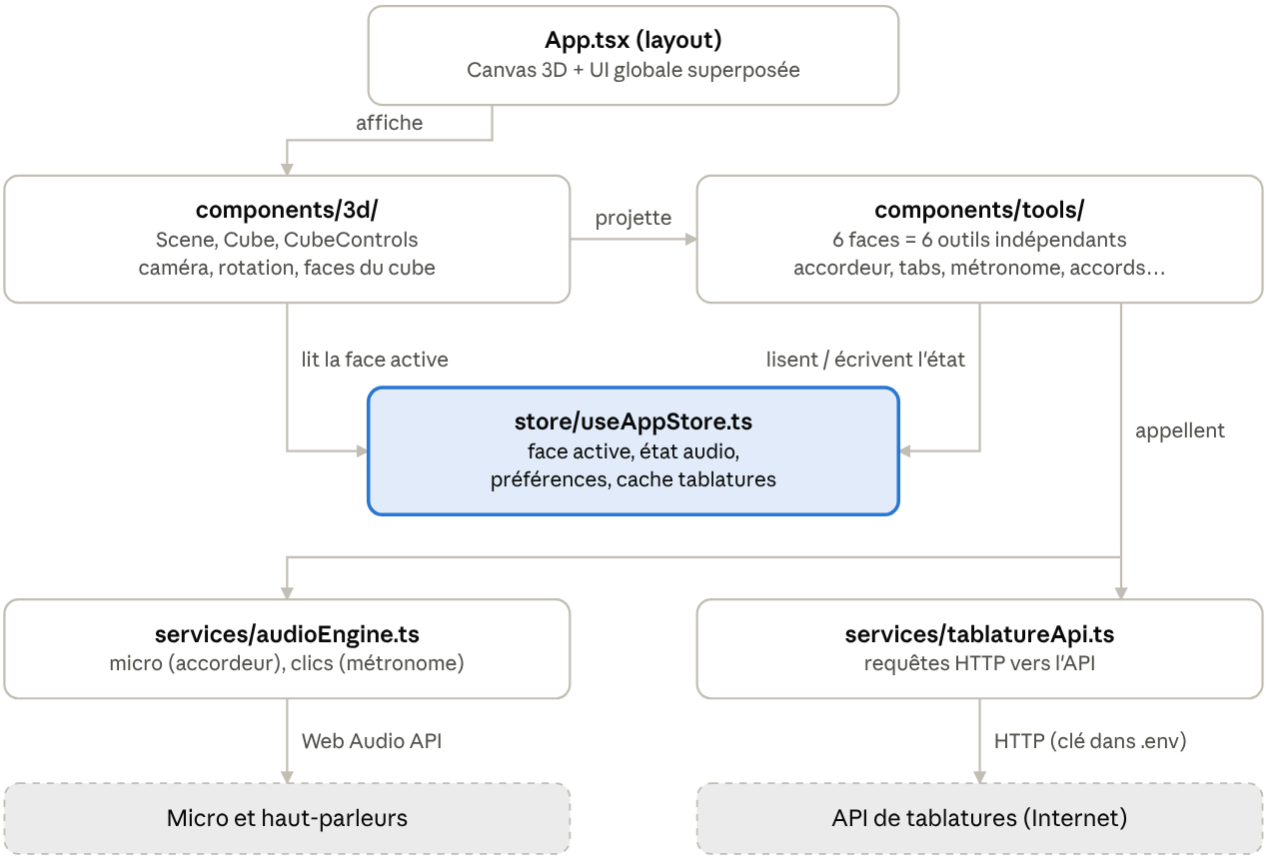
Le code qui touche au matériel ou au réseau est sorti des composants. Les composants restent simples à lire, et un service peut servir à plusieurs outils.

- `audioEngine.ts` regroupe la Web Audio API : capture du micro et analyse pour l'accordeur, synthèse des clics pour le métronome. Un seul moteur audio pour toute l'appli, car le navigateur limite les contextes audio.
- `tablatureApi.ts` regroupe les requêtes HTTP vers l'API de tablatures. L'URL et la clé viennent du `.env` : changer d'API ne touche que ce fichier.

Flux de données

Les outils et le cube ne s'appellent jamais directement : ils passent par le store, et seuls les services touchent au micro ou à Internet.

Le store relie cube et outils ; seuls les services sortent de l'appli



flux de données · 4 couches

Une flèche signifie « utilise ». Exemple : un clic sur « Métronome » change la face active dans le store, le cube la lit et pivote, puis la face Métronome appelle `audioEngine.ts` pour jouer les clics.

Organisation Git et déploiement

Le code passe toujours par une branche de fonctionnalité, puis par `dev`, et n'arrive sur `main` (donc en ligne) qu'une fois stable.

Branche	Rôle	Qui y pousse
`main`	Version stable, déployée automatiquement sur Vercel	Personne directement : uniquement par merge depuis `dev`
`dev`	Intégration du travail des deux membres	Par Pull Request relue par l'autre membre

Branche	Rôle	Qui y pousse
`feature/...`	Une fonctionnalité (ex. `feature/accordeur`)	Son auteur, librement

Pourquoi : deux personnes qui poussent sur la même branche se gênent et cassent la version en ligne. Les branches de fonctionnalité isolent le travail en cours, et la Pull Request oblige chacun à relire le code de l'autre, ce qui compte aussi pour l'évaluation en binôme.

Déploiement : Vercel lit le repo GitHub, détecte Vite, lance `npm run build` et publie le dossier `dist/`. Les variables du `.env.example` se renseignent dans *Project Settings → Environment Variables*.

Répartition des rôles

Répartition fixée le 25/10/2026 : E = Benjamin, I = son collègue, E + I = en binôme. Chacun prend trois outils ; la scène 3D et le déploiement se font ensemble.

Partie	Dossier / fichiers	Responsable
Scène 3D, cube, rotation	`components/3d/`	E + I
État global	`store/useAppStore.ts`	I
Accordeur + moteur audio	`FaceTuner.tsx`, `services/audioEngine.ts`	E
Métronome	`FaceMetronome.tsx`	I
Tablatures + API	`FaceTabs.tsx`, `services/tablatureApi.ts`	E
Dictionnaire d'accords	`FaceChordDictionary.tsx`	E
Carnet d'entraînement	`FacePracticeNotebook.tsx`	I
Réglages	`FaceSettings.tsx`	I
Déploiement Vercel	configuration Vercel	E + I